

MODULE 5 · STORAGE

Access Modes & Reclaim Policies

Who else can mount it, and what happens to the data when you're done

The wrong access mode caps how far your app can scale

A PersistentVolumeClaim binds to a PV with a fixed set of **access modes** and a **reclaim policy** — both decided when the PV is created, not renegotiated later.

Get the access mode wrong and a Deployment that scales fine on CPU and memory stalls on storage: extra replicas land on other nodes and simply can't attach the volume.

BY THE END YOU CAN

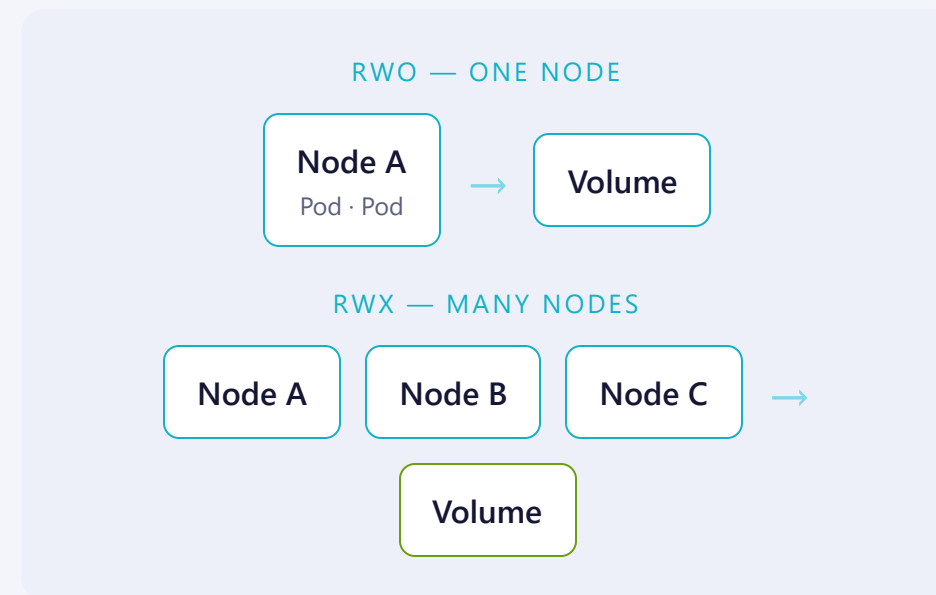
- Explain RWO vs ROX vs RWX vs RWOP
- Predict what happens when a PVC is deleted
- Say why an RWO volume blocks horizontal scaling
- Choose Retain vs Delete for a workload

CONCEPT

Access mode counts nodes, not pods

An access mode says how many **nodes** can mount a volume at once, and whether that mount is read-write or read-only. It says nothing about how many **pods** — several pods on the same node can happily share one ReadWriteOnce volume.

Most block-style storage backends only ever support ReadWriteOnce. Read-write from many nodes (RWX) needs a file-style backend, and far fewer of those exist.



One growing question: how many nodes?

MODE	NODES	PODS	TYPICAL BACKING STORAGE
<code>ReadWriteOnce</code> RWO	One node	Many, on that node	Block-style storage — the default almost everywhere
<code>ReadOnlyMany</code> ROX	Many nodes	Many, read-only	File/object-style storage
<code>ReadWriteMany</code> RWX	Many nodes	Many, read-write	File-style storage — far fewer backends support it
<code>ReadWriteOncePod</code> RWOP	One node	Exactly one pod	Block-style storage, strict single-writer

SET ON THE PV, REQUESTED BY THE PVC

A PVC can only bind to a PV whose access modes it requests — the PV author decides what's possible, the claim just asks for it.

Two fields decide sharing and cleanup

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-rwo
  labels:
    type: local
spec:
  capacity:
    storage: 500Mi
  accessModes:
    - ReadWriteOnce          # one node
  persistentVolumeReclaimPolicy: Retain # keep data
  hostPath:
    path: /tmp/pv-rwo-data
```

- **accessModes** — always a list, even with one entry; a PV can advertise more than one
- **persistentVolumeReclaimPolicy** — **Retain** keeps the data, **Delete** removes the PV (and often the backing storage) with the PVC

The lab's companion PV, `pv-rox.yaml`, is identical except for `ReadOnlyMany` and `Delete`.

No imperative shortcut here — so dry-run earns its keep

OBJECTS WITH AN IMPERATIVE FORM

```
kubectl create deployment web --image=nginx
```

Fine to prototype and throw away

PERSISTENTVOLUMES & PVCS

No `kubectl create pv` — capacity, access modes and reclaim policy are hand-written

Precise settings you don't want to guess at

THE BRIDGE TRICK, STORAGE EDITION

Validate a claim against the API — including whether a matching PV exists — without actually binding it:

```
$ kubectl apply -f pvc-rwo.yaml --dry-run=server
```

SEE IT

Same delete, two different outcomes

RETAIN — KUBECTL DELETE PVC PVC-RWO

Bound



Released

data kept · claimRef blocks reuse

DELETE — KUBECTL DELETE PVC PVC-RWX

Bound



PV removed

backing storage usually removed too

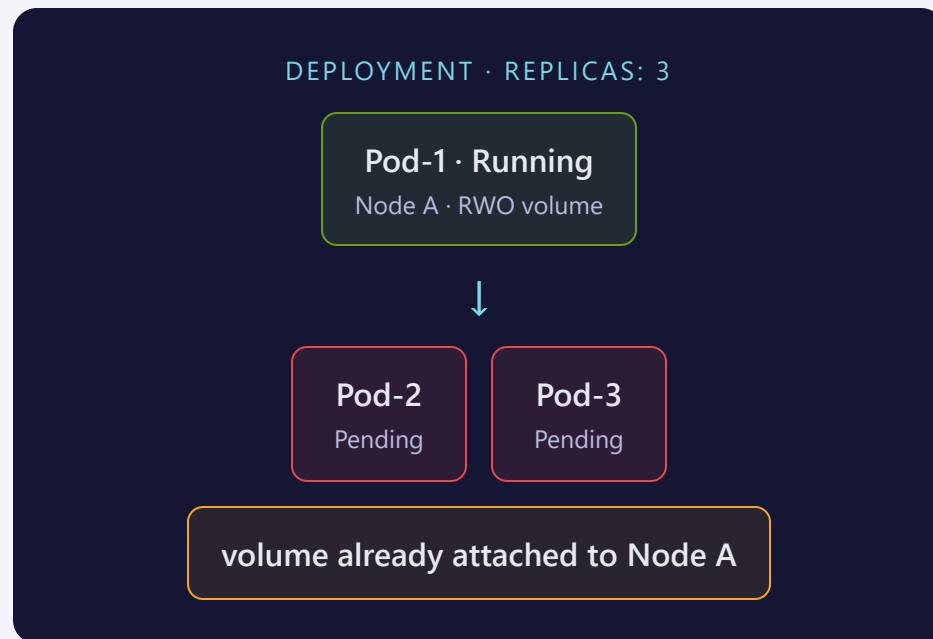
`kubectl get pv` after each delete shows the difference directly in the STATUS column — no guessing required.

THE PUNCHLINE

Scale the Deployment, not the volume

Scale a Deployment whose Pods share one **RWO** PVC, and the extra replicas try to schedule onto other nodes. The volume is already attached read-write to the first node — Kubernetes won't attach it a second time.

The new Pods sit **Pending** with a FailedAttachVolume event, forever.
Fix: give each replica its own volume (StatefulSet + volumeClaimTemplates, Lab 5.4), or move to storage that genuinely supports RWX.



Where people trip

- **Assuming RWX is available.** Most clusters' default storage only supports RWO — check the backend before designing for many-writer access.
- **Deleting a PVC on a Retain PV.** It goes Released, not Available — the `claimRef` must be cleared before anything can bind it again.
- **hostPath + Delete.** The PV object disappears, but the host directory can remain — Delete isn't a guarantee for every backend.
- **Scaling an RWO-backed Deployment.** Extra replicas don't get a second volume — they go Pending, not round-robin across nodes.

Commands to keep at hand

```
$ kubectl get pv                                # ACCESS MODES + RECLAIM POLICY columns
$ kubectl get pv -o custom-
columns=NAME:.metadata.name,ACCESS:.spec.accessModes,RECLAIM:.spec.persistentVolumeReclaimPolicy,STATUS:.status.phase
$ kubectl describe pv NAME                      # claimRef, events
$ kubectl get pvc -n training                   # Bound / Pending
$ kubectl patch pv NAME -p '{"spec":{"claimRef":null}}' # reuse a Released PV
```

Tip: a Released PV isn't broken — it's waiting for you to decide what happens to the old data.

RECAP

Access modes & reclaim policies in one breath

- Access mode = how many **nodes**, not pods, can mount a volume
- RWO (one node) is the default; RWX (many nodes) needs a file-style backend
- Retain keeps data and needs manual cleanup; Delete removes the PV — and often the storage — with it
- An RWO volume caps horizontal scale: one node holds the write lock

HANDS-ON

Now run Lab 5.5
[labs/5.5/](#)

Next: [6.1 — Scheduling](#)